

**Université de Sherbrooke**  
**Faculté de génie**  
**Département de génie informatique**

# **Rapport**

## **Systeme d'exploitation**

Par:  
Langevin, Clovis - Lanc0902  
Gratton, Francis - Graf2102

Présenté à:  
L'équipe professorale

Remis le 4 aout 2026

# 1 Introduction

Ce rapport présente l'analyse et l'amélioration d'une application de conversion d'images utilisant plusieurs fils d'exécution et processus. L'objectif est d'identifier les problèmes liés à la synchronisation des ressources partagées, d'implémenter des mécanismes permettant de corriger ces problèmes et d'évaluer l'impact du parallélisme sur les performances de l'application.

## 2 Identification des problèmes

Dans le code de base de la problématique, plusieurs problèmes liés à la synchronisation et à la gestion des ressources ont été identifiés. Tout d'abord, la variable `task_queue`, utilisée pour stocker les tâches à exécuter, était accessible simultanément par plusieurs threads sans aucun mécanisme de protection. Cette situation pouvait entraîner des conditions de course, provoquant des comportements imprévisibles tels que des accès concurrents, des pertes de données ou des corruptions de la file de tâches. De plus, le système de cache présent dans le programme était initialisé mais non-utilisé, ce qui empêchait son fonctionnement adéquat et pouvait compromettre les performances attendues. Un autre problème concernait la fin du programme, les threads demeuraient en attente même lorsque la file de tâches était vide, empêchant ainsi le programme de se terminer correctement. Enfin, le mécanisme de récupération des tâches reposait sur une écoute continue, où les threads vérifiaient continuellement si de nouvelles tâches avaient été ajoutées à la file. Cette approche entraîne une consommation inutile des ressources processeur.

## 3 Mise en œuvre des mécanismes de synchronisation

Afin de corriger les différents problèmes de synchronisation présents dans l'application, plusieurs mécanismes ont été ajoutés. Tout d'abord, un **mutex** a été utilisé pour protéger les accès à la variable `task_queue`, puisqu'elle est partagée entre plusieurs threads. Ce mécanisme garantit qu'un seul thread à la fois peut accéder ou modifier la file de tâches, éliminant ainsi les conditions de course et assurant l'intégrité des données. Ensuite, le système de cache a été implémenté dans la fonction `processQueue`, permettant de conserver les résultats déjà calculés et d'éviter des traitements redondants, ce qui améliore les performances globales de l'application. Afin de permettre au programme de se terminer correctement, une variable **atomique** a été ajoutée pour maintenir

le nombre de tâches actuellement en cours d'exécution. Contrairement à une variable classique, une variable atomique peut être modifiée simultanément par plusieurs threads sans nécessiter de mutex, tout en garantissant la cohérence de sa valeur. Lorsque cette variable indique qu'aucune tâche n'est en cours et que la file de tâches est vide, les threads peuvent être arrêtés proprement, évitant ainsi que le programme demeure bloqué en attente. Finalement, une **variable de condition** a été introduite afin d'éliminer l'écoute continu. Au lieu de vérifier continuellement si de nouvelles tâches sont disponibles dans `task_queue`, les threads se mettent en veille lorsqu'aucune tâche n'est présente et sont automatiquement réveillés lorsqu'une nouvelle tâche est ajoutée. De cette manière on réduit considérablement l'utilisation inutile du processeur tout en améliorant l'efficacité.

## 4 Gestion des processus

Afin d'améliorer les performances de l'application, la conversion des images a été séparée en plusieurs processus indépendants. Le lancement des processus est effectué à l'aide d'un script Python utilisant le module `multiprocessing`. Chaque processus créé exécute une instance indépendante du programme de conversion d'images et reçoit une série de tâches provenant du générateur de tâches.

Le choix d'utiliser plusieurs processus permet de répartir la charge de calcul sur plusieurs cœurs du processeur. Contrairement aux fils d'exécution partageant la même mémoire, les processus possèdent chacun leur propre espace mémoire, ce qui permet d'isoler les différentes exécutions et d'éviter certains problèmes de synchronisation liés aux données partagées.

Le nombre de processus lancés est configurable afin de pouvoir analyser l'impact du parallélisme sur les performances. Pour chaque configuration testée, un nombre différent de processus et de fils d'exécution a été utilisé. Par exemple, une configuration de 4 processus avec 4 fils crée un total potentiel de 16 unités d'exécution concurrentes permettant de traiter plusieurs images simultanément.

Le script Python utilisé permet donc de contrôler précisément le niveau de parallélisme de l'application et de mesurer l'impact du nombre de processus et de fils sur le temps total de conversion.

## 5 Analyse de la performance

Afin d'évaluer l'impact du parallélisme, plusieurs configurations de processus et de fils d'exécution ont été testées. Les résultats sont présentés dans les tableaux suivants.

| Nombre de processus | 1 fil    | 2 fils   | 3 fils   | 4 fils   |
|---------------------|----------|----------|----------|----------|
| 1                   | 0m4.282s | 0m2.249s | 0m1.500s | 0m1.178s |
| 2                   | 0m2.196s | 0m1.180s | 0m0.865s | 0m0.692s |
| 3                   | 0m1.499s | 0m0.855s | 0m0.645s | 0m0.568s |
| 4                   | 0m1.254s | 0m0.697s | 0m0.583s | 0m0.536s |

Tableau 1. – Temps d'exécution selon le nombre de processus et de fils

| Nombre de processus | 1 fil    | 15 fils  | 48 fils  | 96 fils  |
|---------------------|----------|----------|----------|----------|
| 1                   | 0m4.282s | 0m0.619s | 0m0.508s | 0m0.500s |
| 15                  | 0m0.565s | 0m0.542s | 0m0.539s | 0m0.548s |

Tableau 2. – Temps d'exécution avec un nombre élevé de processus et de fils

Le premier tableau montre qu'une augmentation du nombre de processus et de fils permet de réduire considérablement le temps d'exécution. En passant d'une exécution séquentielle (1 processus et 1 fil) à 4 processus avec 4 fils, le temps de traitement passe de 4.282 secondes à 0.536 secondes. Cette amélioration provient du fait que plusieurs tâches peuvent être exécutées simultanément sur les différents cœurs du processeur.

Cependant, l'ajout de processus et de fils supplémentaires ne garantit pas toujours une amélioration des performances. Le deuxième tableau montre qu'une configuration utilisant 15 processus peut ralentir l'exécution. À ce niveau, les ressources matérielles du système deviennent la limitation principale et la surcharge liée à la gestion des fils supplémentaires réduit les gains possibles. Il est aussi possible de voir un plateau de performance vers 0.5 seconds puisque créer plus de fils ne va pas accélérer les opérations de conversion.

La meilleure configuration testée est donc de 1 processus avec 96 fils, cela s'explique par le fait que le système d'exploitation est capable de répartir ces fils sur les différents cœurs du processeur. Cette approche évite la surcharge supplémentaire liée à la gestion de plusieurs espaces mémoire associés aux processus.

## 6 Conclusion

Les modifications apportées ont permis de corriger les problèmes de synchronisation présents dans l'application tout en améliorant son efficacité grâce à

l'utilisation de mécanismes adaptés tels que les mutex, les variables atomiques et les variables de condition. L'analyse des performances démontre que l'ajout de parallélisme réduit significativement le temps d'exécution, mais qu'un nombre trop élevé de processus ou de fils peut entraîner une saturation des ressources du système d'exploitation.